UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

Sistemas Distribuidos

Clase 12 — SLOs, error budgets, experimentos bajo falla, incidentes y postmortems

Departamento de Computación
Ingeniería Informática – Comisión A – Pilar
Equipo docente – 2026

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

¿Cuándo anda suficientemente bien?

Sistemas Distribuidos

Dónde nos quedamos

- Clase 10: cómo **ver** lo que pasa dentro del sistema.
- Clase 11: cómo decidir de antemano de qué manera **falla**.

Dónde nos quedamos

- Clase 10: cómo **ver** lo que pasa dentro del sistema.
- Clase 11: cómo decidir de antemano de qué manera **falla**.

La pregunta que falta

¿Cuándo está el sistema funcionando **suficientemente bien**?

Dónde nos quedamos

- Clase 10: cómo **ver** lo que pasa dentro del sistema.
- Clase 11: cómo decidir de antemano de qué manera **falla**.

La pregunta que falta

¿Cuándo está el sistema funcionando **suficientemente bien**?

Sin una respuesta explícita, cada discusión se resuelve por intuición, por jerarquía o por quién argumenta mejor:

- ¿Liberamos esta versión hoy?
- ¿Esta alerta merece despertar a alguien a las 3 AM?
- ¿El *circuit breaker* que configuramos alcanza?
- ¿El incidente de ayer fue grave?

Los seis temas de hoy

1. **SLI, SLO y SLA**: definir con números qué es "suficientemente bien".
2. **Error budget**: cuánta falla se tolera, y cómo se usa para decidir.
3. **Alertas sobre el objetivo (*burn rate*)**: avisar a tiempo, sin falsas alarmas.
4. **Experimentos bajo falla (*Chaos Engineering*)**: comprobar antes de un incidente real que el sistema se comporta como suponemos.
5. **Gestión de incidentes**: responder de forma coordinada.
6. **Postmortems**: aprender para que no se repita.

Los seis temas de hoy

1. **SLI, SLO y SLA**: definir con números qué es "suficientemente bien".
2. **Error budget**: cuánta falla se tolera, y cómo se usa para decidir.
3. **Alertas sobre el objetivo (*burn rate*)**: avisar a tiempo, sin falsas alarmas.
4. **Experimentos bajo falla (*Chaos Engineering*)**: comprobar antes de un incidente real que el sistema se comporta como suponemos.
5. **Gestión de incidentes**: responder de forma coordinada.
6. **Postmortems**: aprender para que no se repita.

Para el trabajo práctico

En Asteria Online la garantía no lleva números. Los números de hoy (SLO, error budget) van en la **solución** y en su **trade-off**. Lo vemos al final.

La confiabilidad es gestión de riesgo

El objetivo podría ser: que el sistema no falle nunca.

La confiabilidad es gestión de riesgo

El objetivo podría ser: que el sistema no falle nunca. Pasado cierto punto, más confiabilidad es **peor**:

- **El costo no es lineal**: cada mejora puede costar cien veces más que la anterior. Réplicas, capacidad ociosa y, sobre todo, ingenieros que no construyen funcionalidades.
- **El usuario no lo nota**: acostumbrado a un servicio confiable al 99 %, nadie distingue un servicio al 99,99 % de uno al 99,999 %.
- **Compite con la velocidad de cambio**: más estabilidad, menos cambios.

La confiabilidad es gestión de riesgo

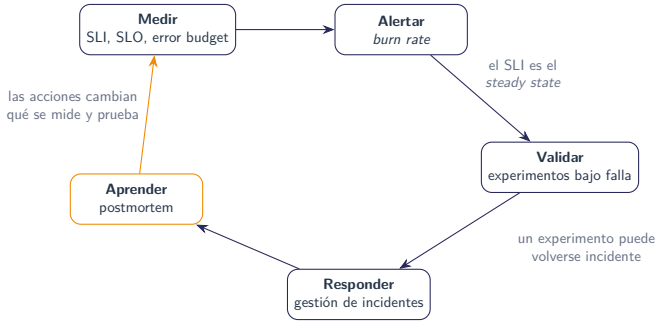
El objetivo podría ser: que el sistema no falle nunca. Pasado cierto punto, más confiabilidad es **peor**:

- **El costo no es lineal**: cada mejora puede costar cien veces más que la anterior. Réplicas, capacidad ociosa y, sobre todo, ingenieros que no construyen funcionalidades.
- **El usuario no lo nota**: acostumbrado a un servicio confiable al 99 %, nadie distingue un servicio al 99,99 % de uno al 99,999 %.
- **Compite con la velocidad de cambio**: más estabilidad, menos cambios.

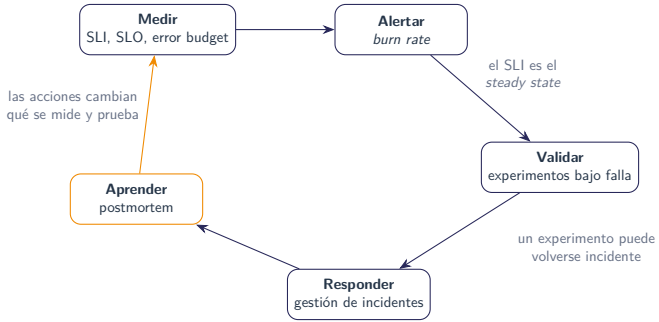
El objetivo es mínimo y máximo a la vez

Por debajo, los usuarios sufren. Por encima, nadie nota el nueve de más, y ese esfuerzo podría haberse usado en otra cosa.

Cómo se encadenan los temas

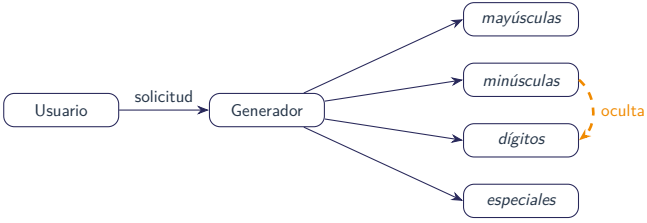


Cómo se encadenan los temas



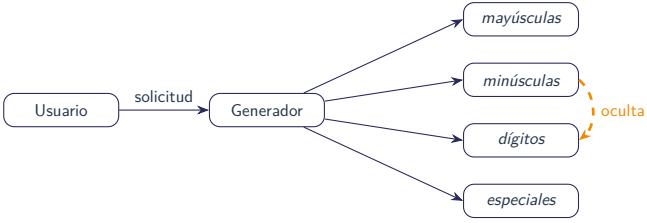
Medir define “normal”; alertar lo vigila; validar y responder lo usan como referencia; aprender lo corrige.

El ejemplo de hoy: el generador de contraseñas



- Lo de la clase 11: 5 llamadas a cada hijo, **20 por solicitud**. *minúsculas* llama por dentro a *dígitos*. Una contraseña sin especiales **no** es una degradación aceptable.
- Supuesto para las cuentas: **5 solicitudes/s** $\Rightarrow$ 18 000 por hora y unas **12,96 millones en 30 días** $(5 \times 60 \times 60 \times 24 \times 30)$.

El ejemplo de hoy: el generador de contraseñas



- Lo de la clase 11: 5 llamadas a cada hijo, **20 por solicitud**. *minúsculas* llama por dentro a *dígitos*. Una contraseña sin especiales **no** es una degradación aceptable.
- Supuesto para las cuentas: **5 solicitudes/s** $\Rightarrow$ 18 000 por hora y unas **12,96 millones en 30 días** $(5 \times 60 \times 60 \times 24 \times 30)$.

Para quien lo usa, "funcionar bien" es: **recibir una contraseña válida** y **recibirla rápido**. Todavía vago: hay que convertirlo en números.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Medir: SLI, SLO y SLA

Sistemas Distribuidos

Una promesa implícita

Todo servicio lleva una promesa: que responda, rápido, y bien. Mientras sea implícita, cada uno la lee a su manera:

- quien desarrolla: “anda bien si no hay errores en los logs”;
- quien lo usa: “si la página de registro no se traba”;
- quien lo vende: “si nadie se queja”.

Una promesa implícita

Todo servicio lleva una promesa: que responda, rápido, y bien. Mientras sea implícita, cada uno la lee a su manera:

- quien desarrolla: “anda bien si no hay errores en los logs”;
- quien lo usa: “si la página de registro no se traba”;
- quien lo vende: “si nadie se queja”.

Un acuerdo explícito tiene tres piezas

1. **Qué se mide**: el indicador (SLI).
2. **Qué valor es aceptable**: el objetivo (SLO).
3. **Qué pasa si no se cumple**: el acuerdo con consecuencias (SLA).

SLI, SLO y SLA

Las tres siglas empiezan con **SL**, *Service Level*: el **nivel de servicio**, es decir, qué tan bien le funciona el servicio a quien lo usa.

- **SLI** (*Service Level Indicator*): una medida cuantitativa del nivel de servicio.
- **SLO** (*Service Level Objective*): un valor objetivo para un SLI: "SLI $\geq$ objetivo".
- **SLA** (*Service Level Agreement*): un contrato con consecuencias si no se cumplen sus SLOs.

SLI, SLO y SLA

Las tres siglas empiezan con **SL**, *Service Level*: el **nivel de servicio**, es decir, qué tan bien le funciona el servicio a quien lo usa.

- **SLI** (*Service Level Indicator*): una medida cuantitativa del nivel de servicio.
- **SLO** (*Service Level Objective*): un valor objetivo para un SLI: "SLI $\geq$ objetivo".
- **SLA** (*Service Level Agreement*): un contrato con consecuencias si no se cumplen sus SLOs.

En el generador

- **SLI**: el porcentaje de solicitudes que reciben una contraseña válida.
- **SLO**: ese porcentaje es al menos **99,9 %** en los últimos 30 días.
- **SLA**: si en un mes queda por debajo de **99,5 %**, el cliente recibe un descuento (ilustrativo).

El SLI como proporción de eventos buenos

$$\text{SLI} = \frac{\text{eventos buenos}}{\text{eventos válidos}} \times 100\,\%$$

- Todos los SLIs en la misma escala: de 0 % a 100 %, donde 100 % es lo mejor.
- Mide bien las fallas **parciales**, que en un sistema distribuido son la norma.
- Disponibilidad = solicitudes exitosas / válidas: la **disponibilidad semántica** de la clase 8, no el *uptime* del proceso.

El SLI como proporción de eventos buenos

$$\text{SLI} = \frac{\text{eventos buenos}}{\text{eventos válidos}} \times 100\,\%$$

- Todos los SLIs en la misma escala: de 0 % a 100 %, donde 100 % es lo mejor.
- Mide bien las fallas **parciales**, que en un sistema distribuido son la norma.
- Disponibilidad = solicitudes exitosas / válidas: la **disponibilidad semántica** de la clase 8, no el *uptime* del proceso.

"Válidos" define la población

Una solicitud mal formada (4xx) no dice nada sobre la salud del servicio: se excluye.

Especificación e implementación

Especificación: lo que le importa al usuario. “La proporción de solicitudes de generación de contraseña que reciben una contraseña válida”.
Implementación: dónde y cómo se mide.

Especificación e implementación

Especificación: lo que le importa al usuario. "La proporción de solicitudes de generación de contraseña que reciben una contraseña válida".
Implementación: dónde y cómo se mide.

Dónde	Ventaja	Limitación
Logs del generador	Barata, desde el día uno.	No ve lo que nunca llegó al proceso.
Balanceador	Más cerca del usuario.	No sabe si la contraseña cumple la política.
Cliente sintético (*prober*)	Recorrido completo, valida contenido.	Tráfico artificial.
Instrumentación en el cliente	Lo más real.	Cara; mezcla la red del usuario.

Regla: medir lo más cerca del usuario que se pueda. Para empezar, lo que cueste poco.

Qué medir según el tipo de servicio

Tipo de servicio	SLIs habituales
Atiende solicitudes	Disponibilidad, latencia, calidad (¿estaba degradada?)
Almacenamiento	Latencia, disponibilidad, durabilidad
Pipeline de datos	Frescura, corrección, cobertura, *throughput*
Cualquiera	Corrección: ¿la respuesta era la correcta?

Qué medir según el tipo de servicio

Tipo de servicio	SLIs habituales
Atiende solicitudes	Disponibilidad, latencia, calidad (¿estaba degradada?)
Almacenamiento	Latencia, disponibilidad, durabilidad
Pipeline de datos	Frescura, corrección, cobertura, *throughput*
Cualquiera	Corrección: ¿la respuesta era la correcta?

La corrección entra en el SLI

Si "bueno" fuera cualquier HTTP 200, una contraseña sin especiales devuelta por un *fallback* mal decidido contaría como éxito. El evento bueno es: **200 con una contraseña que cumple la política**.

SLO: cómo elegir el valor

Empezar por lo que le importa al usuario, no por lo que es fácil de medir.

SLO: cómo elegir el valor

Empezar por lo que le importa al usuario, no por lo que es fácil de medir.

- **No copiar el desempeño actual**: puede atar al equipo a esfuerzos heroicos.
- **Mantenerlo simple**: un "índice de salud" que promedia disponibilidad, latencia y CPU puede no moverse aunque todo cambie. Un SLI por propiedad.
- **Evitar absolutos**: "siempre disponible" no es un requisito.
- **Pocos SLOs**: si uno nunca sirve para ganar una discusión de prioridades, sobra.
- **La perfección puede esperar**: empezar laxo y ajustar.

SLO: cómo elegir el valor

Empezar por lo que le importa al usuario, no por lo que es fácil de medir.

- **No copiar el desempeño actual**: puede atar al equipo a esfuerzos heroicos.
- **Mantenerlo simple**: un "índice de salud" que promedia disponibilidad, latencia y CPU puede no moverse aunque todo cambie. Un SLI por propiedad.
- **Evitar absolutos**: "siempre disponible" no es un requisito.
- **Pocos SLOs**: si uno nunca sirve para ganar una discusión de prioridades, sobra.
- **La perfección puede esperar**: empezar laxo y ajustar.

La ventana

Deslizante de **30 días** (o 4 semanas): se parece a la memoria del usuario. Una ventana fija se reinicia de golpe; una de 90 días permite gastar el 90 % del presupuesto en un día y cumplir igual.

Margen de seguridad y no sobrecumplir

Margen de seguridad

Usar internamente un SLO **más estricto** que el que se publica: da tiempo a corregir un problema antes de que lo note el usuario.

Margen de seguridad y no sobrecumplir

Margen de seguridad

Usar internamente un SLO **más estricto** que el que se publica: da tiempo a corregir un problema antes de que lo note el usuario.

No sobrecumplir. Los usuarios construyen sobre lo que el servicio **realmente** da. Si rinde mucho mejor que su SLO, otros dependen de él **como si no pudiera fallar**, y cuando falla no saben funcionar sin él.

Margen de seguridad y no sobrecumplir

Margen de seguridad

Usar internamente un SLO **más estricto** que el que se publica: da tiempo a corregir un problema antes de que lo note el usuario.

No sobrecumplir. Los usuarios construyen sobre lo que el servicio **realmente** da. Si rinde mucho mejor que su SLO, otros dependen de él **como si no pudiera fallar**, y cuando falla no saben funcionar sin él.

Tres formas de no sobrecumplir

- Diseñarlo para que **no sea más rápido** cuando tiene poca carga.
- **Rechazar a propósito** una pequeña parte de las solicitudes.
- La más drástica: **apagarlo a propósito** de vez en cuando, si no hubo caídas reales. Se usa en servicios de infraestructura de los que dependen muchos otros.

SLA: el acuerdo

La prueba

¿Qué pasa si no se cumple? Si no hay una consecuencia explícita (reintegro, penalidad), es un SLO.

- Es una decisión comercial y legal. Al equipo técnico le toca decir cuán difícil es cumplirlo y que sus SLIs se puedan medir sin discusión.
- Conviene prometer poco: cuantos más clientes tiene un SLA, más difícil es cambiarlo.

SLA: el acuerdo

La prueba

¿Qué pasa si no se cumple? Si no hay una consecuencia explícita (reintegro, penalidad), es un SLO.

- Es una decisión comercial y legal. Al equipo técnico le toca decir cuán difícil es cumplirlo y que sus SLIs se puedan medir sin discusión.
- Conviene prometer poco: cuantos más clientes tiene un SLA, más difícil es cambiarlo.

SLA $<$ SLO interno

El generador promete 99,5 % en el SLA y se exige 99,9 % en el SLO. El equipo actúa al apartarse del 99,9 %, mucho antes de que se active la penalidad.

No todo servicio tiene SLA. Con o sin él, se opera con SLIs y SLOs.

Los objetivos del generador

SLI de disponibilidad, medido en el balanceador:

$$\frac{\text{respuestas 200 con una contraseña que cumple la política}}{\text{todas las solicitudes} - \text{respuestas 4xx}} = \frac{\text{2xx buenas}}{\text{2xx} + \text{5xx}}$$

Las 5xx incluyen los *timeouts* y las conexiones cortadas (502, 504): son las fallas del servicio.

Los objetivos del generador

SLI de disponibilidad, medido en el balanceador:

$$\frac{\text{respuestas 200 con una contraseña que cumple la política}}{\text{todas las solicitudes} - \text{respuestas 4xx}} = \frac{\text{2xx buenas}}{\text{2xx} + \text{5xx}}$$

Las 5xx incluyen los *timeouts* y las conexiones cortadas (502, 504): son las fallas del servicio.

SLI de latencia: el p99 de las solicitudes válidas, medido en el balanceador.

Los objetivos del generador

SLI de disponibilidad, medido en el balanceador:

$$\frac{\text{respuestas 200 con una contraseña que cumple la política}}{\text{todas las solicitudes} - \text{respuestas 4xx}} = \frac{\text{2xx buenas}}{\text{2xx} + \text{5xx}}$$

Las 5xx incluyen los *timeouts* y las conexiones cortadas (502, 504): son las fallas del servicio.

SLI de latencia: el p99 de las solicitudes válidas, medido en el balanceador.

SLOs

Disponibilidad $\geq$ **99,9 %** y p99 de latencia $<$ **500 ms**, sobre 30 días deslizantes.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Error budget

Error budget: cuánto se puede fallar

Si el SLO no es 100 %, hay una falla tolerada: el **error budget** es $1 - \text{SLO}$.

SLO	Caída en 30 días
99 %	7,2 h
99,5 %	3,6 h
99,9 %	43,2 min
99,95 %	21,6 min
99,99 %	4,32 min
99,999 %	25,9 s

Error budget: cuánto se puede fallar

Si el SLO no es 100 %, hay una falla tolerada: el **error budget** es $1 - \text{SLO}$.

SLO	Caída en 30 días
99 %	7,2 h
99,5 %	3,6 h
99,9 %	43,2 min
99,95 %	21,6 min
99,99 %	4,32 min
99,999 %	25,9 s

- Cada nueve divide el presupuesto por diez.
- En solicitudes, el generador (99,9 %) puede fallar **12 960 solicitudes en 30 días**: el 0,1 % de 12,96 millones.
- O sea, **18 por hora** en promedio.

Para qué sirve: alinear incentivos

Desarrollo quiere velocidad; operaciones, estabilidad. ¿Cuánto probar? ¿Cada cuánto desplegar? ¿Cuánto dura el *canary* (clase 9)?

Para qué sirve: alinear incentivos

Desarrollo quiere velocidad; operaciones, estabilidad. ¿Cuánto probar? ¿Cada cuánto desplegar? ¿Cuánto dura el *canary* (clase 9)?

1. Producto define el SLO.
2. El monitoreo mide la disponibilidad real: un tercero neutral.
3. La diferencia es el margen de falla que queda.
4. **Mientras quede margen, se despliega.**

Para qué sirve: alinear incentivos

Desarrollo quiere velocidad; operaciones, estabilidad. ¿Cuánto probar? ¿Cada cuánto desplegar? ¿Cuánto dura el *canary* (clase 9)?

1. Producto define el SLO.
2. El monitoreo mide la disponibilidad real: un tercero neutral.
3. La diferencia es el margen de falla que queda.
4. **Mientras quede margen, se despliega.**

El efecto

El equipo se regula solo: con presupuesto amplio, arriesga; casi agotado, es el propio equipo el que pide más pruebas. Las caídas ajenas (la red, un centro de datos) también lo gastan: la responsabilidad es compartida.

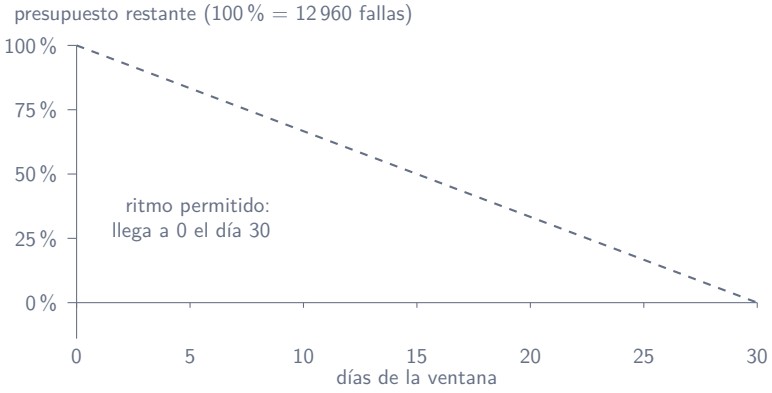
La política de error budget

Las consecuencias se acuerdan **antes**, por escrito, con quién decide si hay desacuerdo. Una política posible para el generador:

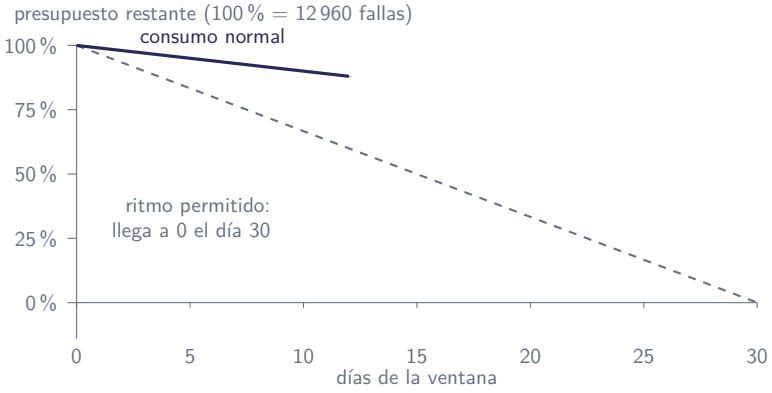
Presupuesto	**Qué se hace**
Más del 50 %	Despliegues normales: siempre con *canary* y reversión automática.
Entre 50 % y 0 %	Despliegues en horarios de poco tráfico, y escalonados: 5 % del tráfico, 15 minutos mirando métricas, siguiente escalón. Más lento y tedioso.
Agotado	Se congela todo lo que no sea confiabilidad o seguridad.

Y una regla más: un incidente que consume más del 20 % del presupuesto exige un postmortem.

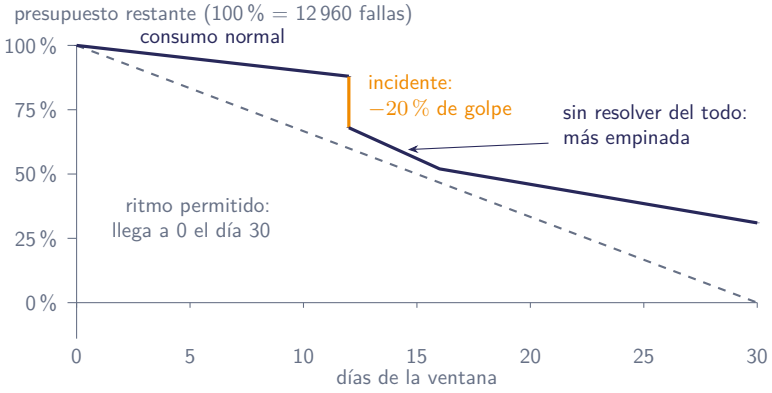
Cómo se gasta el presupuesto



Cómo se gasta el presupuesto

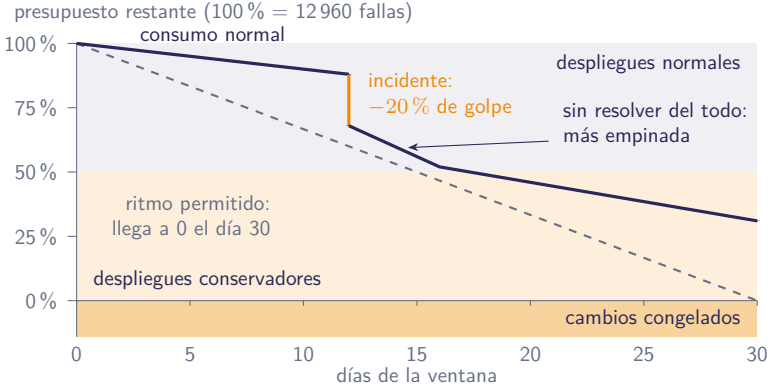


Cómo se gasta el presupuesto



La **pendiente** de la línea es el **burn rate**: cuanto más empinada, más rápido se agota.

Cómo se gasta el presupuesto



La **pendiente** de la línea es el **burn rate**: cuanto más empinada, más rápido se agota. Las bandas son la política de error budget de la próxima slide.

El costo de cada nueve

Monetario: subir de 99,9 % a 99,99 % en un servicio que factura 1 millón de dólares gana 0,09 %: $1\,\text{M} \times 0{,}0009 = 900$ dólares. Si el nueve cuesta más, no conviene.

El costo de cada nueve

Monetario: subir de 99,9 % a 99,99 % en un servicio que factura 1 millón de dólares gana 0,09 %: $1\,\mathrm{M} \times 0{,}0009 = 900$ dólares. Si el nueve cuesta más, no conviene.
No monetario: el usuario ya pierde solicitudes por su propia conexión (su proveedor, el celular, el Wi-Fi). Esa tasa de error de base está entre **0,01 %** y **1 %**. Por debajo, el usuario no distingue los errores del servicio de los de su red.

El costo de cada nueve

Monetario: subir de 99,9 % a 99,99 % en un servicio que factura 1 millón de dólares gana 0,09 %: $1\,\text{M} \times 0{,}0009 = 900$ dólares. Si el nueve cuesta más, no conviene.
No monetario: el usuario ya pierde solicitudes por su propia conexión (su proveedor, el celular, el Wi-Fi). Esa tasa de error de base está entre **0,01 %** y **1 %**. Por debajo, el usuario no distingue los errores del servicio de los de su red.

Idea central

El SLO no es una aspiración: es una **frontera de decisión**. El 100 % casi nunca es el objetivo correcto: es imposible, es caro y el usuario no lo nota.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Alertar sobre el SLO

Síntomas, no causas

El monitoreo tradicional alerta sobre lo fácil de medir: CPU $> 80\,\%$, memoria, disco. Son causas *posibles*.

Síntomas, no causas

El monitoreo tradicional alerta sobre lo fácil de medir: CPU $> 80\,\%$, memoria, disco. Son causas *posibles*.

- La CPU sube por un *backup* o el *garbage collector*: falsos positivos.
- Una alarma que suena seguido sin consecuencias se ignora: **fatiga de alertas** (*alert fatigue*). El día que importa, nadie la mira.

Síntomas, no causas

El monitoreo tradicional alerta sobre lo fácil de medir: CPU $> 80\,\%$, memoria, disco. Son causas *posibles*.

- La CPU sube por un *backup* o el *garbage collector*: falsos positivos.
- Una alarma que suena seguido sin consecuencias se ignora: **fatiga de alertas** (*alert fatigue*). El día que importa, nadie la mira.

Una alerta útil

1. Cuando suena, los usuarios **realmente** tienen un problema.
2. Quien la recibe **puede hacer algo**.

La del SLO cumple la 1 porque vigila el SLI: si cae, hay usuarios afectados, aunque la CPU esté baja. Dice *qué* pasa, no *por qué*: para eso, trazas, métricas y logs (clase 10).

Síntomas, no causas

El monitoreo tradicional alerta sobre lo fácil de medir: CPU $> 80\,\%$, memoria, disco. Son causas *posibles*.

- La CPU sube por un *backup* o el *garbage collector*: falsos positivos.
- Una alarma que suena seguido sin consecuencias se ignora: **fatiga de alertas** (*alert fatigue*). El día que importa, nadie la mira.

Una alerta útil

1. Cuando suena, los usuarios **realmente** tienen un problema.
2. Quien la recibe **puede hacer algo**.

La del SLO cumple la 1 porque vigila el SLI: si cae, hay usuarios afectados, aunque la CPU esté baja. Dice *qué* pasa, no *por qué*: para eso, trazas, métricas y logs (clase 10).

Excepción: alertar por una causa **segura e inminente**, aunque el usuario todavía no lo note: disco casi lleno, certificado por vencer, servicio sin redundancia. Muchas veces, como ticket.

¿Qué vigila una alerta sobre el SLO?

El SLO del generador: 99,9 % en 30 días. Presupuesto: 12 960 fallas. ¿Cuándo suena la alerta?

1. **Cuando el SLO ya no se cumple.**

¿Qué vigila una alerta sobre el SLO?

El SLO del generador: 99,9 % en 30 días. Presupuesto: 12 960 fallas. ¿Cuándo suena la alerta?

1. **Cuando el SLO ya no se cumple.** Llega tarde: no queda nada que proteger.

¿Qué vigila una alerta sobre el SLO?

El SLO del generador: 99,9 % en 30 días. Presupuesto: 12 960 fallas. ¿Cuándo suena la alerta?

1. **Cuando el SLO ya no se cumple.** Llega tarde: no queda nada que proteger.
2. **Cuando el saldo baja de un umbral** (queda menos del 30 %).

¿Qué vigila una alerta sobre el SLO?

El SLO del generador: 99,9 % en 30 días. Presupuesto: 12 960 fallas. ¿Cuándo suena la alerta?

1. **Cuando el SLO ya no se cumple.** Llega tarde: no queda nada que proteger.
2. **Cuando el saldo baja de un umbral** (queda menos del 30 %). Tosca: no distingue gastar el 70 % en tres semanas de gastarlo en una hora; el umbral se vuelve un nuevo cero.

¿Qué vigila una alerta sobre el SLO?

El SLO del generador: 99,9 % en 30 días. Presupuesto: 12 960 fallas. ¿Cuándo suena la alerta?

1. **Cuando el SLO ya no se cumple.** Llega tarde: no queda nada que proteger.
2. **Cuando el saldo baja de un umbral** (queda menos del 30 %). Tosca: no distingue gastar el 70 % en tres semanas de gastarlo en una hora; el umbral se vuelve un nuevo cero.
3. **Según la velocidad de consumo**: con las fallas de la última hora, ¿a qué ritmo se gasta? Si lo agotaría mucho antes de 30 días, actuar ya.

¿Qué vigila una alerta sobre el SLO?

El SLO del generador: 99,9 % en 30 días. Presupuesto: 12 960 fallas. ¿Cuándo suena la alerta?

1. **Cuando el SLO ya no se cumple.** Llega tarde: no queda nada que proteger.
2. **Cuando el saldo baja de un umbral** (queda menos del 30 %). Tosca: no distingue gastar el 70 % en tres semanas de gastarlo en una hora; el umbral se vuelve un nuevo cero.
3. **Según la velocidad de consumo**: con las fallas de la última hora, ¿a qué ritmo se gasta? Si lo agotaría mucho antes de 30 días, actuar ya.

La recomendada es la 3

El SLO define qué es falla y cuánto presupuesto hay; la alerta mide **a qué velocidad se gasta**. Urgencia proporcional al daño.

La alerta del generador, concreta

Tasa de error en una ventana w = solicitudes fallidas en w / solicitudes válidas en w.

Alerta inmediata

tasa de error de la **última hora** $> 1{,}44\,\%$

y

tasa de error de los **últimos 5 minutos** $> 1{,}44\,\%$

La alerta del generador, concreta

Tasa de error en una ventana w = solicitudes fallidas en w / solicitudes válidas en w.

Alerta inmediata

tasa de error de la **última hora** $> 1{,}44\,\%$
y
tasa de error de los **últimos 5 minutos** $> 1{,}44\,\%$

Ejemplo: última hora, 300 fallidas de 18 000 (1,67 %); últimos 5 minutos, 30 de 1 500 (2 %). Suena. A 300 fallas por hora, el presupuesto (12 960) dura **43 horas**.

La alerta del generador, concreta

Tasa de error en una ventana w = solicitudes fallidas en w / solicitudes válidas en w.

Alerta inmediata

tasa de error de la **última hora** $> 1{,}44\,\%$
y
tasa de error de los **últimos 5 minutos** $> 1{,}44\,\%$

Ejemplo: última hora, 300 fallidas de 18 000 (1,67 %); últimos 5 minutos, 30 de 1 500 (2 %). Suena. A 300 fallas por hora, el presupuesto (12 960) dura **43 horas**.

- **Alerta inmediata** (*page*): despierta a la persona de guardia (*on-call*). No puede esperar.
- **Ticket**: tarea para el día hábil siguiente. Se gasta rápido de más, pero no tanto.

¿De dónde sale el 1,44 %? ¿Para qué la segunda ventana? Siguientes slides.

Burn rate

$$b = \frac{e}{1 - s} \qquad \text{se agota en } \frac{T}{b}$$

e: tasa de error · s: objetivo del SLO · T: ventana. Con $b = 1$ se gasta justo el presupuesto.

Burn rate

$$b = \frac{e}{1 - s} \qquad \text{se agota en } \frac{T}{b}$$

e: tasa de error · s: objetivo del SLO · T: ventana. Con $b = 1$ se gasta justo el presupuesto.
Generador: $s = 0{,}999$, $1 - s = 0{,}001$, $T = 720\,\text{h} \Rightarrow b = e/0{,}001$; se agota en $720/b$ horas.

Tasa de error (e)	Burn rate	Se agota en	Situación
0,1 %	1	30 días	justo en el límite
0,6 %	6	5 días	degradación sostenida
1,44 %	14,4	50 h	degradación seria
4,9 %	49	14,7 h	*especiales* al 10 %, un reintento
41 %	410	1,8 h	*especiales* al 10 %, sin reintentos
100 %	1000	43 min	caída total

Elegir la ventana y el umbral

Cuatro propiedades: **precisión** (si suena, hay problema), **cobertura** (todo problema serio suena), **tiempo de detección** y **tiempo de reinicio** (cuánto sigue sonando después de resuelto).

Elegir la ventana y el umbral

Cuatro propiedades: **precisión** (si suena, hay problema), **cobertura** (todo problema serio suena), **tiempo de detección** y **tiempo de reinicio** (cuánto sigue sonando después de resuelto).

- **Una sola ventana no alcanza.** Corta (5 min): rápida pero ruidosa, un pico breve la dispara. Larga (un día): precisa, pero detecta tarde y sigue sonando horas después de resuelto.

Elegir la ventana y el umbral

Cuatro propiedades: **precisión** (si suena, hay problema), **cobertura** (todo problema serio suena), **tiempo de detección** y **tiempo de reinicio** (cuánto sigue sonando después de resuelto).

- **Una sola ventana no alcanza.** Corta (5 min): rápida pero ruidosa, un pico breve la dispara. Larga (un día): precisa, pero detecta tarde y sigue sonando horas después de resuelto.
- **Un solo umbral tampoco.** Si suena solo con $b > 14{,}4$, un consumo sostenido a $b = 10$ nunca la dispara, y agota el presupuesto en **3 días**.

Elegir la ventana y el umbral

Cuatro propiedades: **precisión** (si suena, hay problema), **cobertura** (todo problema serio suena), **tiempo de detección** y **tiempo de reinicio** (cuánto sigue sonando después de resuelto).

- **Una sola ventana no alcanza.** Corta (5 min): rápida pero ruidosa, un pico breve la dispara. Larga (un día): precisa, pero detecta tarde y sigue sonando horas después de resuelto.
- **Un solo umbral tampoco.** Si suena solo con $b > 14{,}4$, un consumo sostenido a $b = 10$ nunca la dispara, y agota el presupuesto en **3 días**.

La solución

Varios umbrales: los rápidos despiertan a alguien, los lentos generan un ticket. Y cada uno exige **dos ventanas a la vez**: la larga confirma que es significativo; la corta, que sigue ocurriendo. La corta apaga la alerta minutos después de resuelto.

Varias ventanas, varios umbrales

Severidad	Ventana larga	Ventana corta	Burn rate	Consumido
page	1 h	5 min	14,4	2 %
page	6 h	30 min	6	5 %
Ticket	3 días	6 h	1	10 %

Varias ventanas, varios umbrales

Severidad	Ventana larga	Ventana corta	Burn rate	Consumido
page	1 h	5 min	14,4	2 %
page	6 h	30 min	6	5 %
Ticket	3 días	6 h	1	10 %

Si un burn rate b dura un tiempo t (acá, la ventana larga), se consume $b \cdot t/T$: $14{,}4 \times 1/720 = 2\,\%$, $6 \times 6/720 = 5\,\%$, $1 \times 72/720 = 10\,\%$. La ventana corta mide la duodécima parte de la larga.

Varias ventanas, varios umbrales

Severidad	Ventana larga	Ventana corta	Burn rate	Consumido
page	1 h	5 min	14,4	2 %
page	6 h	30 min	6	5 %
Ticket	3 días	6 h	1	10 %

Si un burn rate b dura un tiempo t (acá, la ventana larga), se consume $b \cdot t/T$: $14{,}4 \times 1/720 = 2\,\%$, $6 \times 6/720 = 5\,\%$, $1 \times 72/720 = 10\,\%$. La ventana corta mide la duodécima parte de la larga.

Tiempo de detección

La alerta de 1 h suena cuando el promedio de la hora llega a 1,44 %. Con *especiales* al 10 % sin reintentos ($e = 41\,\%$): **unos 2 minutos**. Caída total: **menos de 1 minuto**.

Servicios con poco tráfico

10 solicitudes por hora con SLO de 99,9 %: el presupuesto es de **unas 7 fallas en 30 días**. Una sola falla en una hora es un 10 % de error, burn rate 100, y gasta el **13,9 %** del mes.

Servicios con poco tráfico

10 solicitudes por hora con SLO de 99,9 %: el presupuesto es de **unas 7 fallas en 30 días**. Una sola falla en una hora es un 10 % de error, burn rate 100, y gasta el **13,9 %** del mes.

- Generar tráfico artificial para tener más señal.
- Medir juntos servicios que fallan juntos (comparten una base de datos).
- Cambiar el producto: reintentos del cliente con *backoff* y *jitter*, alternativas.
- Bajar el SLO o agrandar la ventana: decisión de producto.

Contar minutos o contar solicitudes

Qué pasa	**Contando solicitudes** presupuesto: 12 960 fallas	**Contando minutos** malo si falla el 0,1 % o más; presupuesto: 43,2 minutos
Durante 1 hora falla el 1 %: 180 de 18 000.	$180/12\,960 \approx$ **1,4 %** del presupuesto.	Los 60 minutos son malos: **se agota**.

Contar minutos o contar solicitudes

Qué pasa	**Contando solicitudes** presupuesto: 12 960 fallas	**Contando minutos** malo si falla el 0,1 % o más; presupuesto: 43,2 minutos
Durante 1 hora falla el 1 %: 180 de 18 000.	$180/12\,960 \approx$ **1,4 %** del presupuesto.	Los 60 minutos son malos: **se agota**.

Contar minutos **exagera** algunos problemas y **oculta** otros. En un sistema distribuido lo habitual es la degradación parcial (*brownout*).

- **Solicitudes**: para operar el servicio y calcular el burn rate, sobre todo con SLOs de 99,9 % o más.
- **Minutos**: para un SLA o una página de estado (“12 minutos caído este mes”), con mucho tráfico y umbrales pensados para caídas.

Caso: la alerta que nadie creía

Honeycomb: servicio de ingesta de datos de clientes, SLO de **99,995 %**, con alerta de consumo del presupuesto. El equipo todavía **no le creía**.

Caso: la alerta que nadie creía

Honeycomb: servicio de ingesta de datos de clientes, SLO de **99,995 %**, con alerta de consumo del presupuesto. El equipo todavía **no le creía**.

- Una madrugada falla el 1,5 % durante 20 minutos: consumo **300 veces** más rápido que el permitido. Suena a la 1:29.
- La guardia ve el servicio recuperado: "un pico". Recién a la cuarta alerta, a las 9:55, se declara un incidente.
- El monitoreo tradicional (sondas fallidas seguidas) **nunca vio nada**.

Caso: la alerta que nadie creía

Honeycomb: servicio de ingesta de datos de clientes, SLO de **99,995 %**, con alerta de consumo del presupuesto. El equipo todavía **no le creía**.

- Una madrugada falla el 1,5 % durante 20 minutos: consumo **300 veces** más rápido que el permitido. Suena a la 1:29.
- La guardia ve el servicio recuperado: "un pico". Recién a la cuarta alerta, a las 9:55, se declara un incidente.
- El monitoreo tradicional (sondas fallidas seguidas) **nunca vio nada**.

La causa y la lección

Una pérdida de memoria: las máquinas caían y se reiniciaban casi juntas. Quedarse sin memoria era normal; lo anómalo era el efecto sobre los usuarios. Gastaron más de cinco veces el presupuesto, y para ellos la alerta del SLO pasó a ser **la principal**. El SRE Book es más prudente: conserva las alertas por causas inminentes.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Validar: experimentos bajo falla

Sistemas Distribuidos

Por qué no alcanza con probar los componentes

La clase 11 configuró *timeouts*, reintentos y *breakers*. ¿Cómo sabemos que funcionan?

Por qué no alcanza con probar los componentes

La clase 11 configuró *timeouts*, reintentos y *breakers*. ¿Cómo sabemos que funcionan?

La seguridad no se compone

Un cliente con *timeout* de 50 ms llama a dos proveedores: 20 ms en promedio, casi nunca más de 30 ms. Cada uno, solo, es seguro. Los dos en secuencia cumplen *en promedio* (40 ms), pero cuando los dos se demoran un poco a la vez, la suma pasa los 50 ms.

El generador hace **20 llamadas** por solicitud, cada una razonable por separado.

Por qué no alcanza con probar los componentes

La clase 11 configuró *timeouts*, reintentos y *breakers*. ¿Cómo sabemos que funcionan?

La seguridad no se compone

Un cliente con *timeout* de 50 ms llama a dos proveedores: 20 ms en promedio, casi nunca más de 30 ms. Cada uno, solo, es seguro. Los dos en secuencia cumplen *en promedio* (40 ms), pero cuando los dos se demoran un poco a la vez, la suma pasa los 50 ms.

El generador hace **20 llamadas** por solicitud, cada una razonable por separado.

- Las pruebas unitarias y de integración suponen tiempos ideales y *mocks* limpios.
- *Staging* nunca es producción: menos servidores, menos tráfico, red sin congestión.

Chaos Engineering

Definición

Hacer experimentos sobre un sistema para ganar confianza en que soporta condiciones turbulentas en producción. No deduce qué *debería* pasar: provoca la falla de forma controlada y observa qué *pasa*.

Chaos Engineering

Definición

Hacer experimentos sobre un sistema para ganar confianza en que soporta condiciones turbulentas en producción. No deduce qué *debería* pasar: provoca la falla de forma controlada y observa qué *pasa*.

¿Por qué provocar fallas a propósito?

- **Los sistemas pierden margen sin que nadie lo decida**: cada optimización consume holgura. Un pool justo para 5 solicitudes/s anda bien hasta que *especiales* se pone lento.
- **Lo que nunca falla no se sabe manejar**: nadie sabe cómo se comporta el resto sin ese componente.
- **La falla pequeña y frecuente se vuelve rutina**: como un simulacro de evacuación.

Chaos Engineering

Definición

Hacer experimentos sobre un sistema para ganar confianza en que soporta condiciones turbulentas en producción. No deduce qué *debería* pasar: provoca la falla de forma controlada y observa qué *pasa*.

¿Por qué provocar fallas a propósito?

- **Los sistemas pierden margen sin que nadie lo decida**: cada optimización consume holgura. Un pool justo para 5 solicitudes/s anda bien hasta que *especiales* se pone lento.
- **Lo que nunca falla no se sabe manejar**: nadie sabe cómo se comporta el resto sin ese componente.
- **La falla pequeña y frecuente se vuelve rutina**: como un simulacro de evacuación.

Chaos Monkey (Netflix): apaga al azar un servidor de un servicio. Si el servicio no sigue con los demás, el equipo lo corrige.

Un experimento tiene cuatro pasos

1. Definir el ***steady state***: una salida medible que indica comportamiento normal.
2. Formular la **hipótesis**: el *steady state* se mantiene en un grupo de control y en uno experimental.
3. **Inyectar fallas** (*fault injection*) que reflejen eventos reales.
4. Intentar **refutar** la hipótesis: buscar una diferencia entre los grupos.

Un experimento tiene cuatro pasos

1. Definir el ***steady state***: una salida medible que indica comportamiento normal.
2. Formular la **hipótesis**: el *steady state* se mantiene en un grupo de control y en uno experimental.
3. **Inyectar fallas** (*fault injection*) que reflejen eventos reales.
4. Intentar **refutar** la hipótesis: buscar una diferencia entre los grupos.

El steady state es el SLI

Es exactamente lo que construimos al medir. Sin SLIs no hay qué observar. Por eso primero medimos.

Un experimento tiene cuatro pasos

1. Definir el ***steady state***: una salida medible que indica comportamiento normal.
2. Formular la **hipótesis**: el *steady state* se mantiene en un grupo de control y en uno experimental.
3. **Inyectar fallas** (*fault injection*) que reflejen eventos reales.
4. Intentar **refutar** la hipótesis: buscar una diferencia entre los grupos.

El steady state es el SLI

Es exactamente lo que construimos al medir. Sin SLIs no hay qué observar. Por eso primero medimos.

Además: hipótesis sobre lo que se ve desde afuera · fallas que ocurren en la realidad · en producción cuando sea seguro · automatizado · con el daño acotado (*blast radius*).

La hipótesis

Mala

"Queremos ver si aguanta."

No dice qué observar ni qué resultado la refuta.

Buena

"Si *especiales* se pone lenta, las solicitudes afectadas fallan rápido y las demás mantienen su SLI."

Observable desde afuera. Refutable.

La hipótesis

Mala

"Queremos ver si aguanta."

No dice qué observar ni qué resultado la refuta.

Buena

"Si *especiales* se pone lenta, las solicitudes afectadas fallan rápido y las demás mantienen su SLI."

Observable desde afuera. Refutable.

Dos preguntas antes de empezar:

- **¿Se puede medir hoy el *steady state*?** Si el SLI saliera solo de los logs del generador, un experimento que lo tumba no mostraría ninguna falla.
- **¿Qué resultado la refutaría?** Siempre hay fallas por la red del usuario: lo que importa es la **diferencia** con el grupo de control.

Qué inyectar

Apagar servidores. ¿El servicio sigue con los que quedan?

Agregar latencia. Encuentra lo que apagar no muestra: servicios sin alternativa ante un *timeout*, errores que aparecen cuando las respuestas llegan en otro orden.

Hacer fallar una llamada. La solicitud se marca al entrar ("cuando el generador llame a *especiales*, fallar") y la marca viaja como el contexto de una traza (clase 10). Permite afectar una sola llamada, para pocas solicitudes.

Qué inyectar

Apagar servidores. ¿El servicio sigue con los que quedan?

Agregar latencia. Encuentra lo que apagar no muestra: servicios sin alternativa ante un *timeout*, errores que aparecen cuando las respuestas llegan en otro orden.

Hacer fallar una llamada. La solicitud se marca al entrar ("cuando el generador llame a *especiales*, fallar") y la marca viaja como el contexto de una traza (clase 10). Permite afectar una sola llamada, para pocas solicitudes.

Y también: pérdida de paquetes, particiones, CPU, disco.

Probar las dos: rápida y lenta

Una caída es una falla rápida; la latencia, una lenta. Muchos sistemas toleran que una dependencia desaparezca y **muy mal** que se ponga lenta (clase 11).

Al principio, al azar. Después, mirar el árbol de llamadas de una traza y preguntar qué pasa si se quita cada una.

Seguridad del experimento

- Si cada solicitud fallida es irrecuperable (un pago, un dato médico), **no** se experimenta en producción.
- **Acotar el *blast radius***: empezar por "falla una de cada 10 000 solicitudes".
- Poder **seguir una solicitud** entre servicios y conocer la **línea de base** del SLI.
- Tener un **plan de recuperación**: apagar la perturbación no siempre alcanza.
- Escalera: local $\rightarrow$ integración $\rightarrow$ poco tráfico real $\rightarrow$ producción.

Seguridad del experimento

- Si cada solicitud fallida es irrecuperable (un pago, un dato médico), **no** se experimenta en producción.
- **Acotar el *blast radius***: empezar por "falla una de cada 10 000 solicitudes".
- Poder **seguir una solicitud** entre servicios y conocer la **línea de base** del SLI.
- Tener un **plan de recuperación**: apagar la perturbación no siempre alcanza.
- Escalera: local $\rightarrow$ integración $\rightarrow$ poco tráfico real $\rightarrow$ producción.

El experimento consume presupuesto

Perturbar el 1 % del tráfico, y que todo eso falle, es burn rate 10. Treinta minutos: $10 \times 0{,}5/720 \approx$ **0,7 %** del mes. Con poco presupuesto, la política puede **postergar** el experimento.

Criterio de aborto en el idioma de las alertas: se detiene si el burn rate de la ventana corta supera lo esperado.

Dos advertencias

Una prueba de Google que se salió de control

Bloquearon el acceso a **una** base de datos de prueba entre cien. En minutos, usuarios internos y externos perdieron acceso a sistemas clave. Abortaron, pero la **reversión falló**: nunca se había probado.

Dos advertencias

Una prueba de Google que se salió de control

Bloquearon el acceso a **una** base de datos de prueba entre cien. En minutos, usuarios internos y externos perdieron acceso a sistemas clave. Abortaron, pero la **reversión falló**: nunca se había probado.

"Si nadie responde, empiezo"

Avisó por correo que iba a experimentar en el entorno de pruebas. Casi nadie respondió. El entorno quedó inutilizable **una semana**.

Dos advertencias

Una prueba de Google que se salió de control

Bloquearon el acceso a **una** base de datos de prueba entre cien. En minutos, usuarios internos y externos perdieron acceso a sistemas clave. Abortaron, pero la **reversión falló**: nunca se había probado.

"Si nadie responde, empiezo"

Avisó por correo que iba a experimentar en el entorno de pruebas. Casi nadie respondió. El entorno quedó inutilizable **una semana**.

Lección

Probar la reversión antes. El silencio no es consentimiento. Los primeros experimentos, chicos y acordados.

Automatizar y repetir

En el mejor caso, un experimento es **aburrido**: se inyecta la falla y el sistema sigue igual.

Automatizar y repetir

En el mejor caso, un experimento es **aburrido**: se inyecta la falla y el sistema sigue igual.

- Si encuentra una debilidad: corregirla y buscar el mismo tipo de problema en otras partes.
- Después, automatizarlo para que corra periódicamente. Con límites: no apagar el último servidor; no hacer fallar a la vez una dependencia y su alternativa.

Automatizar y repetir

En el mejor caso, un experimento es **aburrido**: se inyecta la falla y el sistema sigue igual.

- Si encuentra una debilidad: corregirla y buscar el mismo tipo de problema en otras partes.
- Después, automatizarlo para que corra periódicamente. Con límites: no apagar el último servidor; no hacer fallar a la vez una dependencia y su alternativa.

Las personas también fallan

Simulacros en los que parte del equipo queda "fuera" un día: aparecen los procesos que dependen de una sola persona. Google los hace periódicamente con el nombre **DiRT** (*Disaster Recovery Testing*), e incluye la práctica de la gestión de incidentes. Siempre con una palabra clave para abortar.

Template del experimento

Escribir el diseño **antes**: obliga a decidir lo que se improvisaría mal, se comparte con los equipos afectados y termina siendo el informe.

Campo	Contenido
Pregunta	Qué mecanismo o supuesto se valida.
Steady state	SLI usado y su valor en la línea de base.
Hipótesis	Qué se espera, escrito de forma refutable.
Perturbación	Qué se inyecta, dónde, cuánto y por cuánto tiempo.
Grupos	Qué tráfico es control y cuál experimental.
Blast radius	Qué porcentaje del tráfico y qué componentes.
Costo esperado	Cuánto error budget se consume.
Criterio de aborto	Qué señal lo detiene y quién lo detiene.
Recuperación	Qué reiniciar, revertir o limpiar.
Resultado	Qué se observó, si la hipótesis se sostuvo, qué acciones siguen.

Un experimento sobre el generador

Pregunta:	¿los *timeouts* y el *breaker* de la clase 11 contienen una dependencia lenta?
Steady state:	SLIs de disponibilidad y latencia, en el balanceador.
Hipótesis:	si las llamadas a *especiales* del 1 % de las solicitudes tardan 2 s más, esas fallan rápido y con error explícito, y el resto mantiene sus SLIs.
Grupos:	latencia solo en las solicitudes marcadas (1 %); el resto es control. 30 minutos.
Costo:	burn rate ≈ 10; $\approx 0{,}7\,\%$ del presupuesto.
Aborto:	burn rate de 5 minutos $> 14{,}4$, o el grupo de control fuera de su línea de base.

Un experimento sobre el generador

Pregunta:	¿los *timeouts* y el *breaker* de la clase 11 contienen una dependencia lenta?
Steady state:	SLIs de disponibilidad y latencia, en el balanceador.
Hipótesis:	si las llamadas a *especiales* del 1 % de las solicitudes tardan 2 s más, esas fallan rápido y con error explícito, y el resto mantiene sus SLIs.
Grupos:	latencia solo en las solicitudes marcadas (1 %); el resto es control. 30 minutos.
Costo:	burn rate ≈ 10; $\approx 0{,}7\,\%$ del presupuesto.
Aborto:	burn rate de 5 minutos $> 14{,}4$, o el grupo de control fuera de su línea de base.

Qué la refutaría

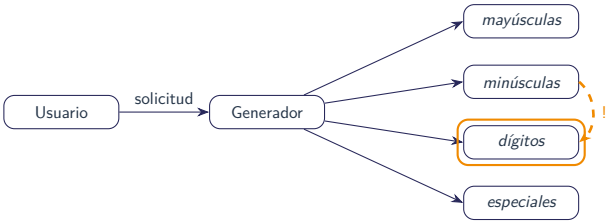
Que suba la latencia del grupo de control: los hilos se comparten (falta un *bulkhead* o el *timeout* es largo). Que las marcadas tarden 2 s en fallar: el *timeout* no está activo.

Lo que no se esperaba

Mismo diseño sobre *dígitos*. Hipótesis: “solo se ven afectadas las llamadas a *dígitos*”.

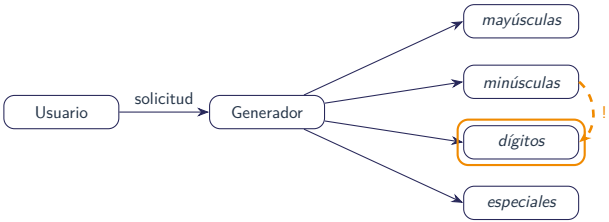
Lo que no se esperaba

Mismo diseño sobre *dígitos*. Hipótesis: “solo se ven afectadas las llamadas a *dígitos*”.



Lo que no se esperaba

Mismo diseño sobre *dígitos*. Hipótesis: "solo se ven afectadas las llamadas a *dígitos*".



Se refuta: las llamadas a *minúsculas* también se vuelven lentas, porque *minúsculas* consulta a *dígitos* por dentro. El experimento descubre la dependencia que no estaba en el diagrama.
Acción: documentarla y revisar que el *timeout* de *minúsculas* contemple el de *dígitos* (clase 11).

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Responder: gestión de incidentes

Sistemas Distribuidos

Qué es un incidente y cuándo declararlo

Un evento que degrada o amenaza el servicio y requiere una **respuesta coordinada**. Una solicitud fallida aislada no lo es.

Qué es un incidente y cuándo declararlo

Un evento que degrada o amenaza el servicio y requiere una **respuesta coordinada**. Una solicitud fallida aislada no lo es.

Condiciones claras, fijadas de antemano

Es un incidente si:

- hace falta involucrar a un **segundo equipo**;
- la caída es **visible para los clientes**;
- sigue sin resolverse tras **una hora** de análisis concentrado.

Qué es un incidente y cuándo declararlo

Un evento que degrada o amenaza el servicio y requiere una **respuesta coordinada**. Una solicitud fallida aislada no lo es.

Condiciones claras, fijadas de antemano

Es un incidente si:

- hace falta involucrar a un **segundo equipo**;
- la caída es **visible para los clientes**;
- sigue sin resolverse tras **una hora** de análisis concentrado.

Con SLOs: una alerta de burn rate *page* suele ser el comienzo de un incidente. Mejor declararlo temprano y cerrarlo rápido que tarde.

El incidente no gestionado

Cae un centro de datos, después otro y otro. La guardia revierte sin éxito, llama a quien escribió el código (3:30 AM en su zona), otros "miran" desde sus terminales, los directivos piden fechas. Alguien despliega una optimización que se le ocurrió en el momento. **Los servidores mueren.**

El incidente no gestionado

Cae un centro de datos, después otro y otro. La guardia revierte sin éxito, llama a quien escribió el código (3:30 AM en su zona), otros "miran" desde sus terminales, los directivos piden fechas. Alguien despliega una optimización que se le ocurrió en el momento. **Los servidores mueren.**

Foco exclusivo en lo técnico: quien sabe arreglar no puede ver el cuadro general.

Mala comunicación: nadie sabe qué hacen los demás.

Trabajo por cuenta propia: cambios con buena intención, sin coordinar.

El incidente no gestionado

Cae un centro de datos, después otro y otro. La guardia revierte sin éxito, llama a quien escribió el código (3:30 AM en su zona), otros "miran" desde sus terminales, los directivos piden fechas. Alguien despliega una optimización que se le ocurrió en el momento. **Los servidores mueren.**

Foco exclusivo en lo técnico: quien sabe arreglar no puede ver el cuadro general.

Mala comunicación: nadie sabe qué hacen los demás.

Trabajo por cuenta propia: cambios con buena intención, sin coordinar.

Todos hacían su trabajo como lo entendían. Falta un **proceso**.

Roles

Tomado de los servicios de emergencia (bomberos, defensa civil): muchas personas que no trabajan juntas a diario.

Rol	Responsabilidad
Incident commander	Estado general, asigna tareas, remueve obstáculos. Tiene los roles que no delegó.
Operaciones	Aplica los cambios. **Único** grupo que toca el sistema.
Comunicación	Actualiza periódicamente al equipo y a los interesados.
Planificación	Lo de largo plazo: registrar, relevos, qué revertir al final.

Roles

Tomado de los servicios de emergencia (bomberos, defensa civil): muchas personas que no trabajan juntas a diario.

Rol	Responsabilidad
Incident commander	Estado general, asigna tareas, remueve obstáculos. Tiene los roles que no delegó.
Operaciones	Aplica los cambios. **Único** grupo que toca el sistema.
Comunicación	Actualiza periódicamente al equipo y a los interesados.
Planificación	Lo de largo plazo: registrar, relevos, qué revertir al final.

Cada uno sabe su rol y no invade el ajeno: eso da **más** autonomía. En un equipo chico, una persona tiene varios roles, pero **explícitos**.

Documento vivo, traspaso y prioridades

- **Puesto de mando**: un lugar o canal donde encontrar al *incident commander*.
- **Documento vivo**: resumen, estado, roles, tareas, línea de tiempo y **criterios de salida**: "dentro de los SLOs durante más de 30 minutos".
- **Traspaso explícito**: "ahora sos el *incident commander*, ¿de acuerdo?".

Documento vivo, traspaso y prioridades

- **Puesto de mando**: un lugar o canal donde encontrar al *incident commander*.
- **Documento vivo**: resumen, estado, roles, tareas, línea de tiempo y **criterios de salida**: "dentro de los SLOs durante más de 30 minutos".
- **Traspaso explícito**: "ahora sos el *incident commander*, ¿de acuerdo?".

El orden

Detener el daño, restablecer el servicio y conservar la evidencia. Primero se mitiga, después se entiende. Durante el incidente no se discuten culpas.

Documento vivo, traspaso y prioridades

- **Puesto de mando**: un lugar o canal donde encontrar al *incident commander*.
- **Documento vivo**: resumen, estado, roles, tareas, línea de tiempo y **criterios de salida**: "dentro de los SLOs durante más de 30 minutos".
- **Traspaso explícito**: "ahora sos el *incident commander*, ¿de acuerdo?".

El orden

Detener el daño, restablecer el servicio y conservar la evidencia. Primero se mitiga, después se entiende. Durante el incidente no se discuten culpas.

La mitigación más frecuente: revertir

En Google, cerca del **70 %** de las caídas viene de un cambio. Un cambio de configuración aplicado a todos los servidores a la vez los hizo caer en bucle; se revirtió en cinco minutos. No había pasado un *canary* completo porque "no era riesgoso".

Runbooks

A favor

En Google, tener los pasos escritos reduce a un tercio el tiempo de recuperación. Se combina con ensayar incidentes pasados.

En contra

Un documento con todos los errores posibles envejece rápido. Uno equivocado es peor que ninguno.

Runbooks

A favor

En Google, tener los pasos escritos reduce a un tercio el tiempo de recuperación. Se combina con ensayar incidentes pasados.

En contra

Un documento con todos los errores posibles envejece rápido. Uno equivocado es peor que ninguno.

Runbook breve del generador

Dueño, guardia, escalamiento. **Depende de**: *mayúsculas*, *minúsculas* (que depende de *dígitos*), *dígitos*, *especiales*.
Dashboards: SLIs, latencia por dependencia, *breakers*.

Alerta "burn rate alto, 1 h": se gastó el 2 % del mes en una hora. Confirmar en el *dashboard*; mirar latencia y errores por dependencia. **Seguro**: revertir el último cambio. **No**: deshabilitar *timeouts*, ni un *fallback* sin especiales. Escalar al dueño de la dependencia.

Aprender: postmortems

Postmortem

Registro escrito del incidente: impacto, cómo se mitigó, causas y **acciones para que no se repita**. No es un castigo.

Postmortem

Registro escrito del incidente: impacto, cómo se mitigó, causas y **acciones para que no se repita**. No es un castigo.

Cuándo es obligatorio

- caída visible sobre un umbral;
- pérdida de datos;
- intervención de la guardia;
- resolución lenta;
- falla del monitoreo;
- más del 20 % del presupuesto.

Qué contiene

- resumen e **impacto** (con error budget);
- **disparador** y **condiciones**;
- resolución y detección;
- **acciones** con dueño;
- qué salió bien, mal, y **dónde hubo suerte**;
- línea de tiempo.

Postmortem

Registro escrito del incidente: impacto, cómo se mitigó, causas y **acciones para que no se repita**. No es un castigo.

Cuándo es obligatorio

- caída visible sobre un umbral;
- pérdida de datos;
- intervención de la guardia;
- resolución lenta;
- falla del monitoreo;
- más del 20 % del presupuesto.

Qué contiene

- resumen e **impacto** (con error budget);
- **disparador** y **condiciones**;
- resolución y detección;
- **acciones** con dueño;
- qué salió bien, mal, y **dónde hubo suerte**;
- línea de tiempo.

"Dónde hubo suerte": lo que salió bien sin que nadie lo garantizara. La próxima vez puede no estar.

Blameless

Buscar las causas **sin acusar** a personas ni equipos. Se supone que todos actuaron con buena intención y con la información que tenían.

Blameless

Buscar las causas **sin acusar** a personas ni equipos. Se supone que todos actuaron con buena intención y con la información que tenían.

- Con culpa, los problemas se esconden.
- Viene de la salud y la aviación: cada error fortalece el sistema.
- No se puede "arreglar" a las personas; sí los sistemas que las ayudan a decidir.

Blameless

Buscar las causas **sin acusar** a personas ni equipos. Se supone que todos actuaron con buena intención y con la información que tenían.

- Con culpa, los problemas se esconden.
- Viene de la salud y la aviación: cada error fortalece el sistema.
- No se puede "arreglar" a las personas; sí los sistemas que las ayudan a decidir.

La pregunta útil

No "¿por qué se equivocó?", sino "¿qué información, presiones y herramientas hicieron que esa decisión pareciera razonable?". Después, todo parece obvio: **sesgo retrospectivo** (*hindsight bias*).

Blameless no es sin responsabilidad: igual señala qué mejorar, apoyado en el sistema.

Disparador y condiciones

Disparador: el evento que lo inició. **Condiciones**: defectos y decisiones previas que esperaban ese evento. Para que un sistema caiga suelen fallar varias defensas a la vez: buscar "la" causa raíz (*root cause*) corrige un solo eslabón.

Disparador y condiciones

Disparador: el evento que lo inició. **Condiciones**: defectos y decisiones previas que esperaban ese evento. Para que un sistema caiga suelen fallar varias defensas a la vez: buscar "la" causa raíz (*root cause*) corrige un solo eslabón.

Incidente ficticio en el generador

Un despliegue cambia la biblioteca HTTP y el *timeout* hacia *especiales* pasa a ser el por defecto: **esperar para siempre**. Días después, *especiales* tiene pausas de *garbage collection*: hilos bloqueados, pool agotado. La alerta suena en 2 minutos; mitigar lleva 25.

Disparador y condiciones

Disparador: el evento que lo inició. **Condiciones**: defectos y decisiones previas que esperaban ese evento. Para que un sistema caiga suelen fallar varias defensas a la vez: buscar "la" causa raíz (*root cause*) corrige un solo eslabón.

Incidente ficticio en el generador

Un despliegue cambia la biblioteca HTTP y el *timeout* hacia *especiales* pasa a ser el por defecto: **esperar para siempre**. Días después, *especiales* tiene pausas de *garbage collection*: hilos bloqueados, pool agotado. La alerta suena en 2 minutos; mitigar lleva 25.

- **Disparador**: las pausas de *especiales*.
- **Latente**: un *timeout* que dependía del valor por defecto.
- **Validación**: el cambio "no era riesgoso", sin *canary*; ningún experimento periódico de latencia.
- **Diagnóstico**: el *dashboard* no mostraba latencia por dependencia.
- **Mitigación**: varios cambios juntos, difíciles de revertir.

Acciones que cambian el sistema

Un postmortem vale lo que valen sus acciones: **un dueño** cada una, **registradas**, **verificables**, que **prevengan** y no solo mitiguen. Cambiar el sistema es más confiable que pedirle a la gente que cambie.

Acciones que cambian el sistema

Un postmortem vale lo que valen sus acciones: **un dueño** cada una, **registradas**, **verificables**, que **prevengan** y no solo mitiguen. Cambiar el sistema es más confiable que pedirle a la gente que cambie.

Malas

"Tener más cuidado al desplegar."
"Mejorar el monitoreo."
Una lista sin dueños ni fechas.

Buenas (generador)

Prevenir: el despliegue falla si falta el *timeout* de una dependencia.
Detectar: experimento semanal de latencia por dependencia.
Detectar: panel de latencia por dependencia.
Mitigar: un solo cambio por despliegue en bibliotecas de red.

Acciones que cambian el sistema

Un postmortem vale lo que valen sus acciones: **un dueño** cada una, **registradas**, **verificables**, que **prevengan** y no solo mitiguen. Cambiar el sistema es más confiable que pedirle a la gente que cambie.

Malas

"Tener más cuidado al desplegar."
"Mejorar el monitoreo."
Una lista sin dueños ni fechas.

Buenas (generador)

Prevenir: el despliegue falla si falta el *timeout* de una dependencia.
Detectar: experimento semanal de latencia por dependencia.
Detectar: panel de latencia por dependencia.
Mitigar: un solo cambio por despliegue en bibliotecas de red.

Y al revés: las acciones propuestas en caliente suelen ser exageradas. Revisarlas con calma.

Un postmortem malo y uno bueno

Malo	Bueno
Términos internos sin explicar.	Lo entiende alguien de otro equipo.
“Afectó a muchos usuarios.”	Impacto medido, con su fuente.
Acciones vagas, sin dueño, misma prioridad.	Dueño, prioridad, estado final verificable.
Señala personas, tono dramático.	Qué falló en el sistema, no quién.
Meses después, solo en el equipo.	En pocos días, compartido.

Un postmortem malo y uno bueno

Malo	Bueno
Términos internos sin explicar.	Lo entiende alguien de otro equipo.
"Afectó a muchos usuarios."	Impacto medido, con su fuente.
Acciones vagas, sin dueño, misma prioridad.	Dueño, prioridad, estado final verificable.
Señala personas, tono dramático.	Qué falló en el sistema, no quién.
Meses después, solo en el equipo.	En pocos días, compartido.

Un postmortem que nadie revisa es como si no existiera

Revisarlo, compartirlo, leerlo en grupo, reconocer a quien manejó bien el incidente. Y una señal de alarma: **incidentes que se repiten**.

Cierre

Aplicación al trabajo práctico: Asteria Online

La garantía no lleva números; la solución, sí

G: “el jugador no debe perder el oro pagado por un combate que no llegó a resolverse”. El SLI y el SLO de cuánto lo logra son de cada **solución**: van en S y en T.

Aplicación al trabajo práctico: Asteria Online

La garantía no lleva números; la solución, sí

G: “el jugador no debe perder el oro pagado por un combate que no llegó a resolverse”. El SLI y el SLO de cuánto lo logra son de cada **solución**: van en S y en T.

La brecha de la Entrega 3, en error budget. SLO de 99,9 % para las instancias de combate (43,2 min en 30 días). Cae un datacenter con un tercio de las instancias:

- Solución A, reubica en 15 min: equivale a 5 min de caída total, **12 %** del presupuesto.
- Solución B, réplicas listas, 1 min: unos 20 s, **menos del 1 %**. A cambio, cuesta más.

Aplicación al trabajo práctico: Asteria Online

La garantía no lleva números; la solución, sí

G: “el jugador no debe perder el oro pagado por un combate que no llegó a resolverse”. El SLI y el SLO de cuánto lo logra son de cada **solución**: van en S y en T.

La brecha de la Entrega 3, en error budget. SLO de 99,9 % para las instancias de combate (43,2 min en 30 días). Cae un datacenter con un tercio de las instancias:

- Solución A, reubica en 15 min: equivale a 5 min de caída total, **12 %** del presupuesto.
- Solución B, réplicas listas, 1 min: unos 20 s, **menos del 1 %**. A cambio, cuesta más.
- **Un SLI por subsistema** (directorio, combate, pagos) y **por tipo de jugador**: priorizar a quienes ya juegan es gastar el presupuesto del ingreso para proteger el de la partida (*load shedding*, clase 11).
- **Validar sin implementar**: ¿qué falla se inyectaría y qué resultado sostendría la decisión CAP elegida?

Números ilustrativos.

Modelo del Curso: Confiabilidad en $D = (P, F, G, S, T)$

P (Problema): Operar con un nivel de servicio explícito, verificar que se cumple bajo falla y aprender cuando no.

F (Factores): Fallas parciales, propiedades emergentes, cambios frecuentes, mediciones imperfectas, poco tráfico, decisiones bajo presión.

G (Garantías): SLOs medidos desde el usuario, alertas proporcionales a la gravedad, hipótesis validadas, respuesta coordinada, acciones que reducen la repetición.

S (Solución): SLI como cociente + SLO de 30 días + error budget con política + burn rate con varias ventanas + experimentos con criterio de aborto + roles + runbooks + postmortems *blameless*.

T (Trade-offs): Cada nueve cuesta más; los experimentos gastan presupuesto; precisión contra detección; la documentación envejece.

Cierre

Idea central de cátedra

Una organización madura no se define por no tener incidentes, sino por medir con objetivos explícitos, validar sus supuestos antes de que lo haga la realidad y aprender cada vez que algo falla.

Cierre

Idea central de cátedra

Una organización madura no se define por no tener incidentes, sino por medir con objetivos explícitos, validar sus supuestos antes de que lo haga la realidad y aprender cada vez que algo falla.

- El SLO convierte "anda bien" en un número; el error budget, en una regla para decidir.
- El burn rate avisa con urgencia proporcional al daño.
- Los experimentos validan la resiliencia de la clase 11, con costo calculado.
- Roles claros reducen el daño; el postmortem *blameless* convierte el incidente en acciones.